ANNADATHA AI

AI-POWERED SMART AGRICULTURE

PROJECT REPORT

Submitted by

KORAPATI HEMANTH	**–**	**241701501**
RASHETHA THASNEEM M	**–**	**241701046**
NITIN R	**–**	**241701041**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND DESIGN





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**ANNADATHA AI-POWERED SMART AGRICULTURE**" is the bonafide work of KORAPATI HEMANTH (241701501) , NITIN R (241701041) and RASHETHA THASNEEM M (241701046) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Certified further that the project contributes to SDG 2, SDG 9 and SDG 12, with a primary focus on SDG 12

Signature of the HOD	Signature of the Supervisor
Dr. S. Vinodh Kumar	Dr. P. Revathy
Professor,The Head of the Department	Professor
Department of Computer Science and Design	Department of Computer Science and Design
Rajalakshmi Engineering College	Rajalakshmi Engineering College
Chennai	Chennai

Submitted for the Phase I Project Viva-Voce examination held on _____.

INTERNAL EXAMINER EXTERNAL EXAMINER

ABSTRACT

Agriculture is increasingly supported by digital technologies, yet farmers may find it difficult to interpret crop symptoms, weather conditions, soil information, and market signals in one place. Annadatha AI is proposed as an AI-powered smart agriculture web application that brings crop-image analysis and farm-context information into a single interface. The project is designed to accept crop photographs and structured details such as crop type, growth stage, cultivated area, planting date, soil characteristics, fertilizer and irrigation practices, and available weather information. An AI analysis service is intended to return a structured assessment covering crop identification, visible disease indications, estimated severity, possible yield outlook, risk factors, and practical recommendations.

The application is developed using a modern web stack based on React, TypeScript, and TanStack Start. Server-side functions are used to validate requests and keep provider credentials out of browser code. The AI integration is designed to try Google Gemini first and use OpenRouter as a fallback when the primary provider is unavailable. Responses must be validated before being shown to users, and failures should produce a friendly message rather than raw provider errors. Additional planned or integrated areas include multilingual UI, location-aware weather, crop and analysis history, market information, and learning-oriented AR/MR experiences.

This Phase I report describes the problem, objectives, related concepts, architecture, requirements, proposed modules, testing approach, risks, and future work. Since external AI models and live data providers can change availability, the report distinguishes intended functionality from results that require confirmation through actual deployment and testing. AI-generated crop advice is decision support only and should not replace local agricultural experts, laboratory soil tests, or verified product-label guidance.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN**, **Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. S. VINODHKUMAR, Ph.D.**, Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work .We convey our sincere and deepest gratitude to our internal guide ,**DR.P.REVATHY.,** Department of Computer Science and Design , for her valuable guidance throughout the course of the project. We are very glad to thank our Project Coordinator, **DR.P.REVATHY.,** Department of Computer Science and Design, for his valuable guidance and useful tips during our reviews to build our project.

TABLE OF CONTENTS

ACKNOWLEDGEMENT
ABSTRACT
LIST OF TABLES
LIST OF FIGURES
LIST OF ABBREVIATIONS
1 INTRODUCTION
1.1 General
1.2 Background
1.3 Problem Statement
1.4 Motivation
1.5 Objectives
1.6 Scope
1.7 Existing System
1.8 Limitations of Existing System
1.9 Proposed System
1.10 Advantages of Proposed System
2 LITERATURE REVIEW
2.1 Introduction
2.2 Review of Related Concepts
2.3 Comparative Analysis
2.4 Research Gap
3THEORETICALBACKGROUNDAND SYSTEM DESIGN
3.1 General
3.2 AI-Assisted Crop Image Analysis
3.3 Prompt Design and Structured Output
3.4 Server Functions and Validation
3.5 Provider Fallback Strategy
3.6 Weather and Location Services
3.7 Multilingual User Interface
3.8 System Architecture
3.9 Data Flow Diagram
3.10 Use Case Overview
3.11 Data and Privacy Considerations
4 METHODOLOGYANDPROJECTDESCRIPTION

4.1 General
4.2 Requirement Analysis
4.3 Functional Requirements
4.4 Non-Functional Requirements
4.5 Technology Selection
4.6 System Modules
4.7 Module Integration
4.8 System Configuration
4.9 Feasibility Study
4.10 Testing Methodology
4.11 Initial Status and Challenges
5CONCLUSIONSANDFUTUREENHANCEMENTS
5.1 Conclusion
5.2 Phase I Status
5.3 Limitations
5.4 Phase II Objectives
5.5 Future Enhancements
5.6 Work Schedule
REFERENCES
CO–SDG RELEVANCE RECORD
PROGRAM OUTCOMES
PROGRAM SPECIFIC OUTCOMES
CO–PO–PSO MAPPING

LIST OF TABLES

Table 1.1 Limitations of existing approaches

Table 2.1 Comparison of agriculture-support approaches

Table 3.1 Core data entities

Table 4.1 Functional requirements

Table 4.2 Non-functional requirements

Table 4.3 Technology stack

Table 4.4 Module descriptions

Table 4.5 Test cases

Table 5.1 Future enhancementsand proposed schedule

LIST OF FIGURES

Figure 3.1 High-level systemarchitecture

Figure 3.2 Crop analysis workflow

Figure 3.3 Use-case overview

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	ArtificialIntelligence
API	Application Programming Interface
AR	Augmented Reality
MR	Mixed Reality
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation Nitrogen,
NPK	Phosphorus and Potassium Global
GPS	Positioning System Hypertext
HTTP	Transfer Protocol Hypertext Transfer
HTTPS	Protocol Secure Internationalization
i18n	Sustainable Development Goal
SDG	Database
DB	Quality Assurance
QA	

CHAPTER 1
INTRODUCTION

1.1 General

Agricultureis affected by crop health, soil properties, irrigation, climate, cultivation practices, and changing market conditions. Farmers often need to consult different sources to understand these factors, and the information may be difficult to interpret without specialist support. Annadatha AI aims to provide a unified web interface through which users can submit crop photographs and relevant farm details, then receive an AI-assisted summary of visible crop concerns and possible next steps.

1.2 Background

Recent advances in computer vision and large language models make it possible to analyse images alongside contextual information. A crop photograph can provide visual evidence of leaf spots, discoloration, wilting, or other visible symptoms, while details such as crop variety, growth stage, soil characteristics, and recent weather can help frame the interpretation. Such predictions remain uncertain: similar symptoms can have different causes, and image quality or missing context can affect the output.

Annadatha AI is conceived as a decision-support application rather than a substitute for field inspection. The interface is intended to combine image upload, farm-profile inputs, AI analysis, weather context, history, and farmer-oriented recommendations. Its server-side architecture also separates browser interaction from AI provider credentials.

1.3 Problem Statement

Farmers may nothaveimmediate access to a crop specialist or a simple digital workflow for interpreting visible crop symptoms together with cultivation context. Existing information sources can be fragmented, and generic advice may not reflect crop stage, soil condition, irrigation, or local weather. The problem addressed by Annadatha AI is the lack of a single, accessible interface that organizes these inputs and presents an understandable, structured AI- assisted assessment while communicating uncertainty and encouraging expert verification.

1.4 Motivation

The projectis motivated by the opportunity to make agricultural information easier to access, especially for users who benefit from visual summaries and multilingual interfaces. A guided form can help users record relevant farm details, while a structured report can make crop-health information easier to review. Provider fallback and clear error handling are intended to improve resilience when an AI service is temporarily unavailable.

1.5 Objectives

1. To design a responsive web application for smart-agriculture information and crop analysis.
2. To accept crop photographs and structured details about the farm, crop, soil, irrigation, and cultivation practices.
3. To integrate AI-assisted image interpretation for crop identification and visible disease indications.

4. To present estimated severity, potential risks, yield outlook, and actionable recommendations with appropriate uncertainty.
5. To design server-side AI integration with Gemini as the primary provider and OpenRouter as a fallback.
6. To validate AI responses and prevent malformed or incomplete data from being presented as reliable findings.
7. To support multilingual UI content and make room for location-aware weather and market information.
8. To maintain a clear separation between verified functionality, demo data, and planned enhancements.

1.6 Scope

1.6.1 Included Scope

- Crop-image upload and validation for supported image formats.
- Collection of crop, farm, soil, fertilizer, irrigation, and weather context where available.
- AI request construction and provider selection on the server.
- Structured analysis output covering visible symptoms, severity estimate, risk, yield outlook, and recommendations.
- User-facing loading, empty, error, and demo states.
- Multilingual interface design and language-aware AI instructions.
- Potential dashboard, crop records, analysis history, weather, and market-information modules.

1.6.2 Scope Limitations

- The AI output is an estimate based on submitted images and details, not a laboratory diagnosis.
- Yield prediction is sensitive to local agronomic data and should not be treated as a guaranteed forecast.
- Live weather and market information depend on external providers and permissions.
- Some optional areas, including AR/MR learning features, may remain planned until implemented and tested.
- The report does not claim measured model accuracy without a documented evaluation dataset.

1.7 Existing System

Agricultural support currently comes from extension services, agronomists, printed guides, agricultural portals, weather applications, market-price services, and crop-disease identification tools. Each source can be useful, but the user may need to move between several interfaces and manually combine information. Image-only diagnosis tools may omit farm context, while general AI chat interfaces may return inconsistent formats or advice that is difficult to compare across analyses.

1.8 Limitations of Existing System

Existing approach	Typical limitation
Manual field inspection	May require accesstoan expert and can take time.
Standalone disease-identification app	May focus on imagelabels without farm-specific context.
Weather application	Provides weather databut may not connect it to crop observations.
Market-price portal	May be separate fromcrop-health and farm records.
General-purpose AI chatbot	Output structure andspecificity may vary; advice needs verification.

1.9 Proposed System

AnnadathaAI proposes acommon interface for crop photographs, cultivation details, AI- assisted analysis, and contextual agricultural information. The application sends validated input to a server-side function, which builds a prompt and calls the selected AI provider. Gemini is attempted first; OpenRouter is used as a fallback when appropriate. The result is normalized and validated against an expected schema before being displayed. If providers fail, the interface should show a friendly unavailable message and offer a clearly labelled demo analysis only when the user explicitly chooses it.

1.10 Advantages of Proposed System

- Combines crop images withstructured farm context in one workflow.
- Uses a consistent result structure for easier reading and comparison.
- Separates secret provider credentials from client-side code.
- Supports a fallback provider to reduce dependence on one external service.
- Can present multilingual explanations for a broader user group.
- Can be extended with history, weather, market information, and learning modules.
- Can clearly distinguish real AI analysis from demo data.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

This chapter reviewsthe main technical areas that inform Annadatha AI: computer vision for plant and leaf imagery, AI-assisted agricultural decision support, structured generation through language models, responsive web interfaces, and location-aware environmental information. The discussion is conceptual and is intended to guide design decisions; it does not claim that a particular model has been independently benchmarked in this project.

2.2 Review of Related Concepts

2.2.1 Computer Vision for Crop Health

Computervisioncan identifyvisual patterns in leaf and crop images, including spots, discoloration, lesions, and deformation. Image classification assigns an image to a category, while object detection or segmentation can localize affected regions. Performance depends on image quality, training data, crop variety, lighting, background, and whether the image shows symptoms at a useful stage. A production-grade diagnosis system should be evaluated using labelled images captured under realistic field conditions.

2.2.2 AI-Based Agricultural Decision Support

Agricultural decision-supportsystems combine observations with crop, soil, climate, and management information to help users evaluate options. Useful recommendations need to be contextual, explain assumptions, and avoid overstating certainty. For example, a visible symptom alone may not distinguish disease from nutrient deficiency, water stress, or chemical injury. The proposed application therefore treats image interpretation as a preliminary signal and should advise expert confirmation when the consequence of a decision is significant.

2.2.3 Large Language Models and Structured Output

Largelanguagemodels cansynthesize imageobservations andtextual context into a readable explanation. However, unrestricted responses may omit fields, invent values, or return inconsistent formatting. Schema-guided prompts and server-side validation help keep output predictable. Validation should reject or mark missing values rather than silently replacing them with fabricated confidence scores or generic findings.

2.2.4 Web Applications and Server-Side AI Integration

Abrowser interfaceoffers cross-deviceaccess withoutinstallinga dedicated desktop program. React supports reusable interface components, TypeScript improves static checking, and TanStack Start provides application routing and server-function capabilities. Provider keys should remain server-side, request inputs should be validated, and user-facing errors should not expose raw provider responses or secrets.

2.2.5 Weather and Market Context

Weather information can help interpret crop stress and plan irrigation, but it must be associated with a clear location and timestamp. Market-price information is similarly dependent on source, market, commodity, unit, and update time. A reliable interface should display provenance and avoid showing fabricated or stale data as current.

2.3 Comparative Analysis

Approach	Strength	Limitation	Relevance to Annadatha AI
Manual expert consultation	Contextual, experienced judgement	May not be immediately available	AIoutput should complement, not replace, experts.
Image-only classifier	Fast visual category prediction	Limited farm context	Combine imagewith structured details.
General-purpose chatbot	Flexible natural-language output	May be inconsistent or ungrounded	Validate a fixed response schema.
Separate weather/market tools	Focused, domain-specific data	Information remains fragmented	Present contextual data in one interface.
Proposed integrated application	Unified workflow and structured output	Depends on provider reliability and input quality	Use validation, source labels, and fallback handling.

2.4 Research Gap

The practical gap addressed by the project is integration. Users may need to combine image-based observations, crop and soil details, weather, and market context across separate services. Annadatha AI aims to make this workflow more coherent through a single interface, structured AI output, and clear distinctions between live information, AI estimates, and demo content. Demonstrating a true research contribution will require formal evaluation of accuracy, usefulness, response time, and usability in realistic field conditions.

CHAPTER 3
THEORETICAL BACKGROUND AND SYSTEM DESIGN

3.1 General

Theproposed system is organized into presentation, validation, provider integration, result normalization, and persistence/context layers. Each layer has a defined responsibility so that a user-interface change does not require rewriting AI-provider logic, and a provider change does not require changing every page that consumes analysis results.

3.2 AI-Assisted Crop Image Analysis

Theuser submits one or more crop photographs along with farm and crop details. The server constructs a prompt that asks the model to distinguish visible evidence from inference, report uncertainty, and return a structured object. The expected analysis may include crop name and confidence, growth stage, health score and status, possible disease, severity, yield outlook, risk factors, recommendations, and farmer advice. These fields represent intended application output; their reliability must be assessed through testing.

3.3 Prompt Design and Structured Output

Auseful promptshould specify thetask,the context fields, the supported output schema, the selected language, and safety constraints. The model should not invent missing soil measurements, weather values, or yield data. If the image is unclear, the result should say that the evidence is insufficient. Chemical-treatment recommendations should not provide unsafe or unsupported dosage instructions; users should follow local agricultural extension guidance and product labels.

3.4 Server Functions and Validation

Server functions receivevalidated request data and perform external API calls without exposing secret keys to the browser. Input validation should enforce allowed image data URLs, a bounded number of images, known provider names, supported languages, and size limits. Output validation should verify required keys and types, check numeric ranges, and reject malformed JSON. Missing fields must be marked unknown or trigger a controlled error rather than being filled with invented scores.

3.5 Provider Fallback Strategy

The primary providerisGoogle Gemini. If the request fails for a recoverable availability or rate-limit reason, the application may retry within a bounded policy. If the primary provider is unavailable or the configured model is not found, the request may be routed to OpenRouter using the configured model slug. Provider errors should be logged on the server with sensitive information removed. If both providers fail, the UI should show a generic message and allow the user to choose Demo Analysis. Demo results must be prominently labelled as demonstration data and never be stored or displayed as real diagnosis without a clear label.

3.6 Weather and Location Services

Location-aware weatherrequires userpermission through the browser geolocation API. Coordinates can be passed to a weather provider, and the result should show the location, observation or forecast time, units, and provider. If permission is denied or the provider fails, the

application should offer manual location entry or a clear unavailable state. It should not substitute a hardcoded city or fake weather values while implying they are live.

3.7 Multilingual User Interface

Internationalization should be centralized so that navigation, buttons, forms, validation messages, empty states, error messages, and dashboard labels change consistently with the selected language. AI prompts should request recommendations in the selected language, while internal data keys remain stable for validation and rendering. At minimum, Tamil and English are relevant initial languages, with additional languages added through translation dictionaries.

3.8 System Architecture

Figure 3.1 presents the high-level architecture. The browser UI gathers user input and sends a request to server-side functions. The server validates the request, calls the configured AI provider, validates the response, and returns a normalized result. Optional weather, history, and market services should be connected through separate interfaces so that failure in one service does not invalidate unrelated features.

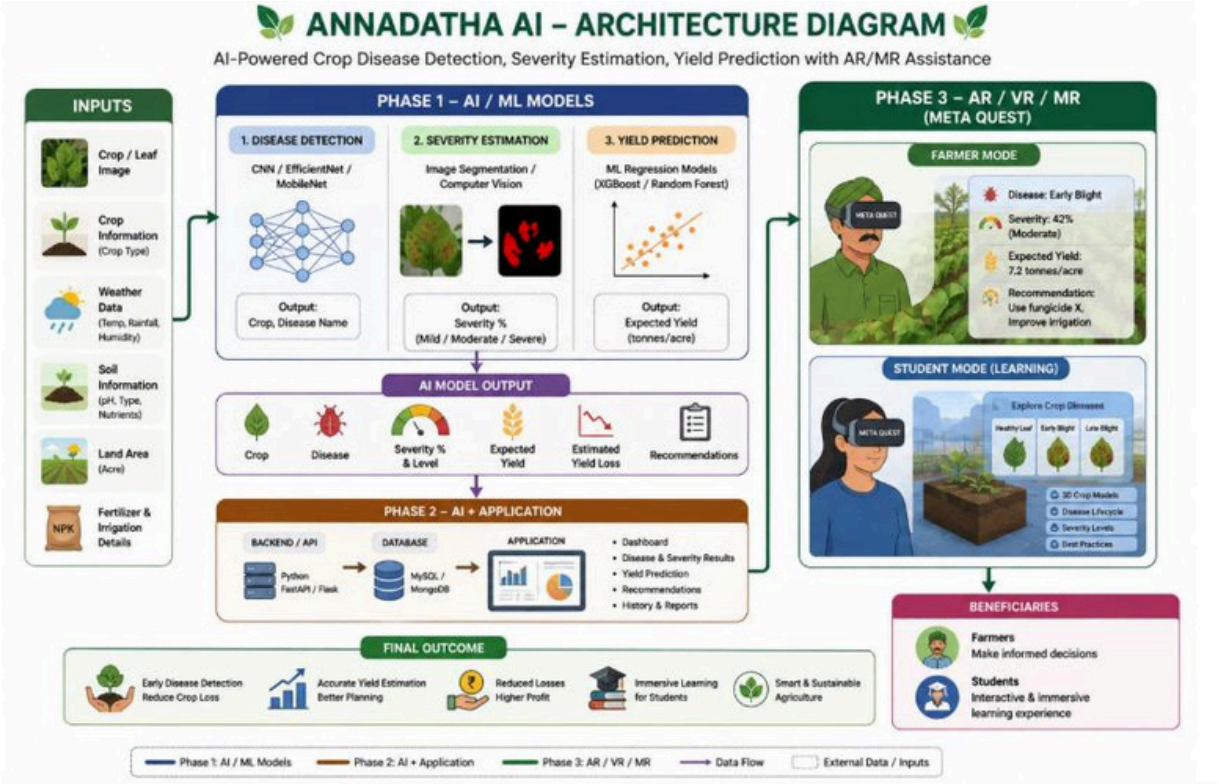


Figure 3.1 High-level system architecture

3.9 Data Flow Diagram

The crop-analysis workflow begins when a user uploads a photograph and enters available farm details. The application validates the input before sending it to the server. The server selects the provider, prepares the prompt, receives the model response, and validates the structured output. The client renders the result and communicates uncertainty or failure states. Figure 3.2 summarizes the flow.

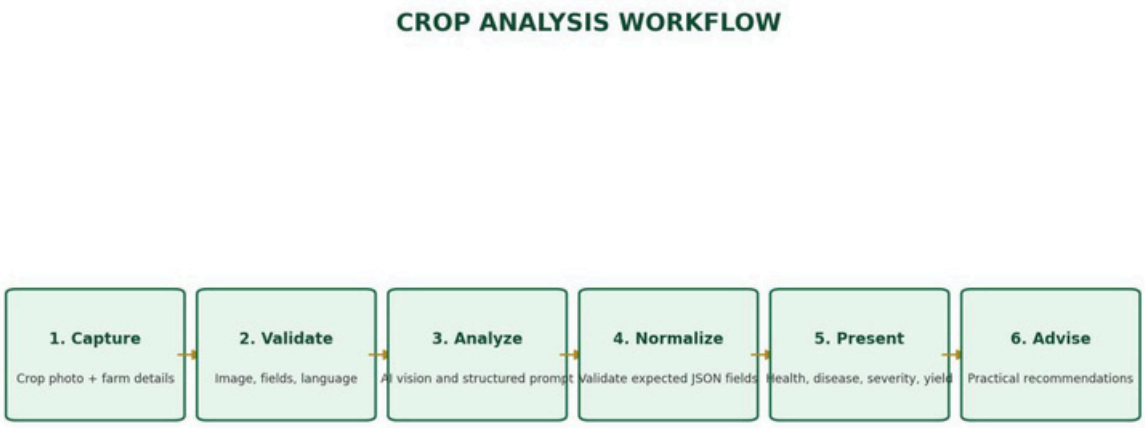


Figure 3.2 Crop analysis workflow

3.10 Use Case Overview

Theprimaryactor isthefarmer or user. The main use cases are maintaining farm details, uploading crop images, requesting analysis, viewing disease and severity estimates, reviewing recommendations, checking weather, viewing analysis history, selecting a language, and consulting market information when a live data source is available.

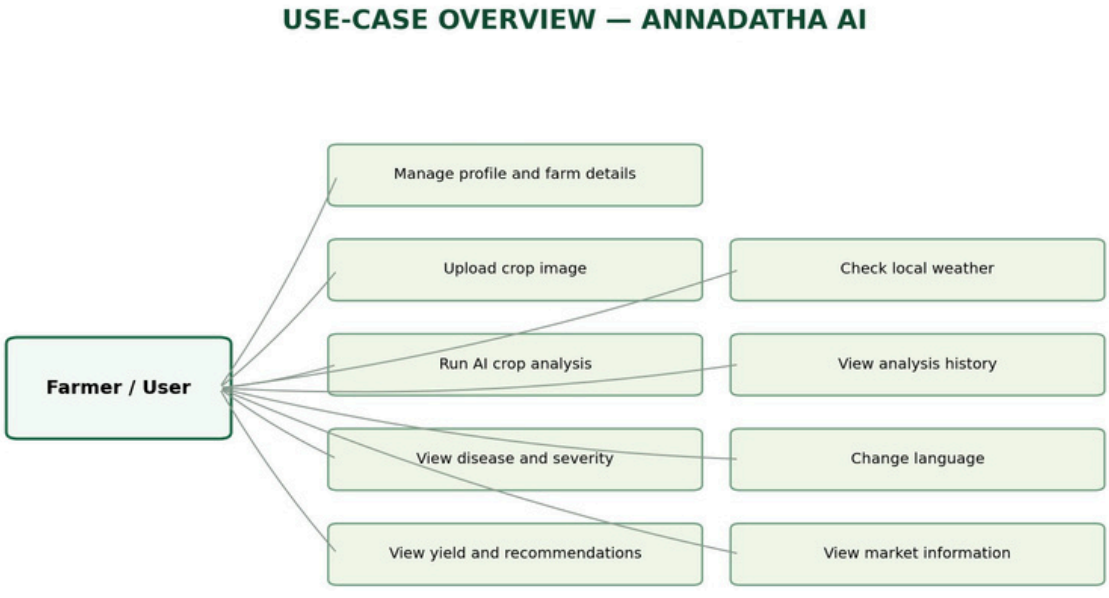


Figure 3.3 Use-case overview

3.11 Core Data Entities

Entity	Example fields	Purpose
User/Farmer	display name,location, language preference	Personalizesthe interface and language.
FarmProfile	farm size, district, state, soil type	Stores farmcontext where persistenceisenabled.
CropRecord	crop type, variety, planting date, growth stage	Representsacultivated crop.
Analysis Request	image references, details, provider, timestamp	Tracks therequest and its inputs.
Analysis Result	crop assessment, disease, severity, risks, advice	Stores normalized analysis if persistence is implemented.
Weather Snapshot	coordinates, temperature, humidity, rainfall, timestamp	Represents sourced weather context.
Market Quote	commodity, market, price, unit, timestamp, source	Represents sourced market information.

3.12 Data and Privacy Considerations

Theapplicationshould collect only information needed for its features. AI keys and privileged database credentials must remain in server-only environment variables and must never be committed to source control. Uploaded crop images and farm profiles should be stored only when the user-facing feature requires it and the retention policy is clear. Logs should avoid image data, tokens, personal details, and raw provider payloads. Authentication and row-level access controls should be used if user-specific records are persisted.

CHAPTER 4
METHODOLOGY AND PROJECT DESCRIPTION

4.1 General

Themethodology follows an incremental web-development approach: identify requirements, design the component and server boundaries, implement the user workflow, connect AI providers, validate outputs, test failure paths, and refine the interface. Features that depend on external providers must be tested separately from static UI components.

4.2 Requirement Analysis

Therequirements are derived from the project goal of making crop analysis and farm information easier to access. The user should be able to provide an image and context, request an analysis, understand the result, and recover gracefully from invalid input or provider failures. The system should not present demo or placeholder data as real information.

4.3 Functional Requirements

ID	Requirement
FR-01	Allow a usertoselect and upload supported crop images.
FR-02	Collect crop,farm, soil, irrigation, fertilizer, and weatherdetails where available.
FR-03	Validate imagecount, file type, form fields, provider selection, and language.
FR-04	Call Gemini asthe primary AI provider from server-side code.
FR-05	Use OpenRouteras a configured fallback provider whenappropriate.
FR-06	Validate modeloutput and render normalized analysis fields.
FR-07	Show friendlyloading, empty, unavailable, and validationstates.
FR-08	Offer explicitlytriggered and clearly labelled demo analysiswhen designed.
FR-09	Support centralized language switching across the interface.
FR-10	Use browser geolocation for weather only with user permission.
FR-11	Provide analysis history, crop records, and market information where implemented.
FR-12	Keep provider credentials and privileged keys out of client bundles.

4.4 Non-Functional Requirements

ID	Requirement	Acceptance consideration
NFR-01	Security	Secrets stay server-side; inputs are validated.
NFR-02	Reliability	Provider failures return controlled errors and do not crash the page.
NFR-03	Usability	Forms and results use clear labels and actionable feedback.
NFR-04	Performance	Image sizes and request durations are bounded.
NFR-05	Maintainability	UI, validation, provider calls, and data access are separated.
NFR-06	Accessibility	Keyboard navigation, contrast, labels, and responsive layouts are considered.
NFR-07	Localization	Language changes apply consistently to visible UI text.
NFR-08	Data integrity	Unknown values remain unknown rather than being fabricated.

4.5 Technology Selection

Technology	Role in project	Reason for selection
React	Component-based UI	Reusable pages, forms, and dashboard components.
TypeScript	Application language	Static checking and clearer data contracts.
TanStack Start	Routing and server functions	Supports route-driven UI and server-side request handling.
Vite	Development/build tooling	Fast development server and frontend build.
Zod	Schema validation	Validates input and structured response shapes.
Gemini API	Primary AI provider	Image and text analysis when configured and available.
OpenRouter API	Fallback AI provider	Alternative model endpoint when configured.
Supabase (if enabled)	Authentication/data persistence	Can support user records and protected data access.
Browser Geolocation	Location input	Requests user permission for current coordinates.

4.6 System Modules

4.6.1 Farmer Profile and Farm Details Module

Thismodule collects farmer and farm context, including location, farm size, crop type, variety, growth stage, planting date, soil characteristics, fertilizer use, irrigation, and pesticide practices. Fields should be optional when a user does not know the value, and the interface should not force fabricated measurements.

4.6.2 Crop Image Upload Module

Theimagemodule lets users select a crop photograph and preview it before submission. It should validate file type and size, limit the number of images, and explain how to capture a useful photo. If image processing or upload fails, the user should retain the entered form values and receive a clear message.

4.6.3 AI Crop Analysis Module

Theanalysismodule constructs aprovider request containing the image and relevant details. It requests structured output for crop identification, visible symptoms, disease possibilities, severity estimate, health status, risk assessment, yield outlook, and recommendations. The result must clearly distinguish observations from model-generated inference.

4.6.4 AI Provider Fallback Module

Theproviderlayer selectsGeminifirst and uses OpenRouter as fallback according to the configured policy. Retry logic should be limited to transient failures; a model-not-found or invalid-configuration response should not be retried repeatedly. Server logs should record provider and status without exposing API keys or unnecessary user data.

4.6.5 Weather and Market Information Module

Weatherand market components shouldonlydisplay current data when a live source is connected and its response is valid. Each data view should identify the location or market, units, timestamp, and source. If the service is unavailable, show an empty state instead of a fabricated value.

4.6.6 Dashboard, Crop Records, and History Module

The dashboard is intended to summarize the user'ssaved crop records and previous analyses. History must be linked to the correct user if authentication is enabled. When no records exist, the interface should show a useful empty state rather than made-up totals, crop cards, or diagnosis history.

4.6.7 Language and Learning Module

The languagemodule centralizestranslations for visible UI content and requests AI recommendations in the selected language. Optional AR/MR or student-learning experiences can explain crop health and sustainable practices; these should be described as future work until their interactions and content are implemented and tested.

4.7 Module Integration

TheUI callsa server function with validated image and context fields. The server function builds the prompt and invokes the provider adapter. The adapter returns raw provider output to a validation layer, which checks the schema and produces a stable response object. The UI renders

that object and maps known error conditions to friendly messages. Optional database persistence should occur only after the response is validated and should enforce access controls.

4.8 System Configuration

4.8.1 Hardware Requirements

Component	Minimumpractical requirement
Computer	Modern laptop or desktop with a current browser
Memory	8 GB RAM recommended for development
Network	Internet connection for AI and live data APIs
Camera	Required only if image capture from camera is supported; file upload is sufficient otherwise

4.8.2 Software Requirements

Software	Purpose
Node.jsLTS	JavaScript runtime for development and build
npm	Dependency installation and scripts
Modernbrowser	Run the web interface
Git	Version control
Vercelorcompatible host	Deployment of the web application
Environment variables	Store server-side API credentials and configuration

Example server-only configuration names include GEMINI_API_KEY, GEMINI_MODEL, OPENROUTER_API_KEY, and OPENROUTER_MODEL. The actual values must be supplied privately through the local environment or deployment settings. Do not place secret values in the report, client code, screenshots, or Git repository.

4.9 Feasibility Study

4.9.1 Economical Feasibility

The application uses a browser interface and common development tools, which reduces the need for specialized hardware. Operational cost depends on hosting, AI provider usage, database services, and weather or market data plans. Free tiers may have rate limits or model availability changes, so cost assumptions should be reviewed before public deployment.

4.9.2 Technical Feasibility

The selected stack supports component-based web development and server-side API calls. Technical risks include dependency compatibility, provider model changes, request limits, large image payloads, and external service outages. These risks can be managed through pinned compatible dependencies, schema validation, bounded retries, monitoring, and deployment tests.

4.9.3 Social Feasibility

Asimpleinterface and Tamil-language support can improve accessibility for some users. The application must avoid presenting uncertain model output as a definitive diagnosis. It should explain limitations and encourage local expert verification for high-impact treatment decisions.

4.10 Testing Methodology

Testing shouldcoverindividual validation functions, provider adapters, integration paths, UI states, and deployment behavior. Because AI responses are probabilistic, tests should verify structure and error handling rather than requiring every natural-language sentence to match exactly.

4.10.1 Unit Testing

Unittests should cover input schema validation, language selection, response normalization, numeric bounds, and error-to-message mapping. Invalid images, missing fields, malformed JSON, and unsupported language codes should be tested.

4.10.2 Integration Testing

Integrationtests should verify that a valid request reaches the server function, the provider adapter receives the expected model and payload, and the normalized response reaches the UI. Mock provider responses can be used in automated tests, but these tests should not be described as live model validation.

4.10.3 Functional and System Testing

Functional tests should verifyimage selection, form submission, loading feedback, successful result rendering, provider fallback, unavailable-service messaging, demo labelling, language switching, geolocation permission handling, and empty history states. System testing should be repeated after dependency updates and before each deployment.

4.10.4 Security Testing

Review the client bundle and repository for secrets; verify that server-only environment variables are used; test authentication and database access rules if persistence exists; ensure raw provider error bodies are not shown to users; and verify that uploads are size-limited. A vulnerable dependency must be upgraded to a patched compatible version rather than bypassing the hosting security block.

4.10.5 Representative Test Cases

Test ID	Scenario	Expected outcome
TC-01	Submitasupported crop imagewith valid fields	Request is accepted and loading state appears.
TC-02	Submitunsupported file type	Validation message appears; no providercallismade.
TC-03	PrimaryAI provider returns a transienterror	Bounded retry or fallback policy is followed.
TC-04	Primaryprovider returns model-not-found	Configuration error is logged and fallbackisattemptedif configured.
TC-05	Both providers fail	Friendly unavailable message is shown.
TC-06	AI returns malformed JSON	Result is rejected or handled safely; fabricated scores are not added.
TC-07	User explicitly selects demo analysis	Result is visibly labelled demo data.
TC-08	User denies geolocation permission	Manual location option or unavailable state appears.
TC-09	User changes language	Visible labels and messages update consistently.
TC-10	No saved crop records exist	Empty state appears instead of fake records.
TC-11	Inspect browser bundle	No secret API key is exposed.
TC-12	Deploy with dependency lockfile	Build installs patched, compatible dependencies.

4.11 Initial Status and Challenges

The project design includes a modern React and TypeScript application, server-side AI integration, a primary/fallback provider strategy, and a structured crop-analysis workflow. During development, provider model availability and dependency compatibility can cause failures even when the UI code is valid. These issues must be addressed through accurate environment configuration, compatible dependency versions, lockfile verification, and testing of the deployed build.

This report intentionally does not claim a measured disease-classification accuracy, yield-prediction error rate, or production uptime because such figures require a documented dataset and repeatable evaluation. The implementation status of optional features such as persistent history, live market prices, weather, and AR/MR learning should be confirmed against the current source code before final submission.

CHAPTER 5

CONCLUSIONS AND FUTURE ENHANCEMENTS

5.1 Conclusion

Annadatha AI proposes a unified web application that helps organize crop images and farm context into a structured AI-assisted assessment. Its core design combines a React and TypeScript interface, server-side request handling, a primary Gemini provider with an OpenRouter fallback, response validation, and user-facing result presentation. The broader goal is to make crop-health information easier to access while clearly communicating uncertainty and the need for expert verification.

The most important engineering principles are secure handling of API credentials, schema validation, bounded retry behavior, clear separation of demo and live results, and truthful display of weather or market information. These principles improve reliability and reduce the risk that incomplete or fabricated information will be mistaken for real agricultural data.

5.2 Current Phase I Status

Area	Phase I statustoverify
Webinterface	React/TypeScript/TanStack Start project structure is identified.
Cropanalysis	AI request and response flow is designed; verify successful livetests.
Gemini/OpenRouter	Provider fallback isintended; confirm model availability and configuration.
Response validation	Ensure required fieldsare checked without fabricated defaults.
Weather and market	Confirm live provider,location, timestamp, and data provenance.
History and crop records	Confirm persistence and user-level access control.
Localization	Confirm every visible UI message is translated.
Deployment	Confirm dependency tree is patched and production build succeeds.

5.3 Limitations

- AImodeloutput can be incorrect, incomplete, or overconfident.
- Similar visual symptoms can have different causes that require field inspection or laboratory tests.
- Yield estimates require local data and validated models to be dependable.
- Provider availability, rate limits, pricing, and model slugs can change.
- Weather and market information depend on external source coverage and freshness.
- Low bandwidth, low-quality images, and incomplete farm details may reduce usefulness.
- Features not yet verified in the current implementation must not be presented as completed.

5.4 Phase II Objectives

9. Createa repeatable evaluation set of crop images with verified labels and crop context.
10. Measure response time, error rates, and agreement with expert assessment.
11. Improve image guidance and explicitly identify insufficient evidence.
12. Test provider fallback and malformed-response handling under controlled failures.
13. Validate live weather and market data sources, timestamps, and units.
14. Complete and test language coverage for all visible interface content.
15. Review security, authentication, database access policies, and deployment configuration.
16. Conduct usability testing with representative users and document feedback.

5.5 Future Enhancements

Enhancement	Expected benefit	Priority
Expert-reviewed crop dataset	More reliable evaluation of disease indications	High
Location-aware weather	Context for crop and irrigation decisions	High
Verified market data	Price awareness withsource and timestamp	Medium
Crop timeline and history	Track changes across analyses	Medium
Tamil and additional languages	Improved accessibility	High
Offline-friendly input drafts	Better use in low-connectivity settings	Medium
AR/MR learning experiences	Interactive crop-health education	Low / exploratory
Notifications and reminders	Timely crop-care prompts	Medium
Explainability and uncertainty display	Helps users interpret model confidence responsibly	High

5.6 Phase II Work Schedule

Period	Planned activity	Deliverable
Weeks1–2	Review code,fix dependencies,establish tests	Stable reproducible build
Weeks3–4	Create labelledevaluation cases and feedback form	Evaluation dataset and protocol
Weeks5–6	Improve outputvalidation and provider fallback	Reliable error and fallback handling
Weeks7–8	Integrate and verify weather/market sources	Sourced data with timestamps
Weeks9–10	Complete localization and accessibility review	Consistent multilingual UI
Weeks11–12	Usability testing, results analysis, final report	Evaluation summary and final demonstration

REFERENCES

[1] React Documentation, “React — The library for web and native user interfaces.” Available: https://react.dev/

[2] TypeScript Documentation, “TypeScript: JavaScript With Syntax For Types.” Available: https://www.typescriptlang.org/docs/

[3] TanStack Documentation, “TanStack Start.” Available: https://tanstack.com/start/latest

[4] Google AI for Developers, “Gemini API Documentation.” Available: https://ai.google.dev/gemini-api/docs

[5] OpenRouter Documentation, “API Reference and Models.” Available: https://openrouter.ai/docs

[6] Zod Documentation, “TypeScript-first schema validation.” Available: https://zod.dev/

[7] Supabase Documentation, “Database, Authentication and Row Level Security.” Available: https://supabase.com/docs

[8] World Meteorological Organization, “Weather, climate and water.” Available: https://wmo.int/

[9] Food and Agriculture Organization of the United Nations, “Digital technologies in agriculture.” Available: https://www.fao.org/digital-agriculture/

[10] United Nations, “Sustainable Development Goals.” Available: https://sdgs.un.org/goals

CO–SDG RELEVANCE RECORD

The following alignment is proposed for academic discussion and should be reviewed against the institution's approved course-outcome and SDG mapping requirements.

SDG	Relevanceto AnnadathaAI	Proposed contribution
SDG2 – Zero Hunger	Crop-health awareness and farm decision support	May help users organizecrop observations and access practical guidance.
SDG 9 – Industry, Innovation and Infrastructure	Use of AI and web technology in agriculture	Demonstrates digital innovation in an agricultural context.
SDG 12 – Responsible Consumption and Production	Context-aware crop and resource recommendations	Could encourage evidence-based use of water and agricultural inputs when validated.

PROGRAM OUTCOMES

The following generic outcome mapping is a draft. Replace it with the exact Program Outcomes (POs), Program Specific Outcomes (PSOs), and Course Outcomes (COs) prescribed by the department.

- PO1 – Engineering knowledge: Apply computing and engineering fundamentals to design a web-based AI application.
- PO2 – Problem analysis: Identify the fragmentation of crop-health and farm-context information.
- PO3 – Design/development: Design a modular system with input validation and provider adapters.
- PO4 – Investigation: Define a test approach for AI outputs and service reliability.
- PO5 – Modern tool usage: Use current web frameworks, validation tools, and AI APIs.
- PO6 – Engineer and society: Consider accessibility and practical agricultural needs.
- PO7 – Sustainability: Relate the application to sustainable agriculture and responsible resource use.
- PO8 – Ethics: Communicate uncertainty and protect user data and API credentials.
- PO9 – Individual and team work: Coordinate implementation, testing, and documentation.
- PO10 – Communication: Present results through structured reports and a readable UI.
- PO11 – Project management: Plan milestones, dependencies, testing, and deployment.
- PO12 – Lifelong learning: Adapt to changing AI provider APIs and software versions.

PROGRAM SPECIFIC OUTCOMES

• PSO1 (draft) – Apply software development and data-handling techniques to build reliable user-facing applications.

• PSO2 (draft) – Integrate AI services into a practical domain workflow while considering validation, usability, and security.

These PSOs are placeholders and must be replaced with the exact outcomes approved for the user's degree programme.

CO–PO–PSO MAPPING

Illustrative mapping only. The final values must be approved by the project guide and aligned with the institution's official CO/PO/PSO definitions.

Course outcome / project outcome	**PO1**	**PO2**	**PO3**	**PO4**	**PO5**	**PO8**	**PO10**	**PSO1**	**PSO2**
CO1: Analyse the agricultural problem	2	3	1	2	1	2	2	2	2
CO2: Design the application architecture	2	2	3	2	3	2	2	3	3
CO3: Integrate and validate AI services	2	2	3	3	3	3	2	3	3
CO4: Test and document the solution	1	2	2	3	2	2	3	2	2

Mapping scale: 1 – Slight (Low); 2 – Moderate (Medium); 3 – Substantial (High). If there is no correlation, use "--". Values above are illustrative and should not be submitted as institution-approved mappings without review.

Signature of the Supervisor